



Directory > ... >

Control and data plane architectural
pattern for Durable Objects

Copy page



Control and data plane architectural pattern for Durable Objects

Last reviewed: 11 months ago

Introduction

[Durable Objects](#) are built on-top of [Cloudflare Workers](#) spanning several locations across our global infrastructure network. Each Durable Object instance has its own durable storage persisted across requests, in-memory state, single-threaded execution, and can be placed in a specific region.

A single Durable Object instance has certain [performance and storage capabilities](#). Therefore, to scale an application without being restricted by the limits of a single instance we need to shard our application data as much as possible, and take advantage of the [Cloudflare infrastructure](#) [↗](#) by spreading our Durable Object instances across the world, moving both the data and compute as close to the users as possible.

This document describes a useful architectural pattern to separate the control plane from the data plane of your application to achieve great performance and reliability without compromising on functionality.

- The **control plane** provides the administrative APIs used to manage resource metadata. For example, a user creating and deleting a wiki, or listing all wikis of a user.
- The **data plane** provides the primary function of the application and handles the operations on the resources data directly. For example, fetching

and updating the content of a wiki, or updating the content of a collaborative document. Data planes are intentionally less complicated and usually handle a much larger volume of requests.

- The **management plane** is an optional component of a system providing a higher level of interaction than the control plane to simplify configuration and operations. In this document, we will not focus on this as the same principles apply as to the control plane.

Control and data plane separation pattern

In this pattern, our application consists of at least one Durable Object instance per resource type handling all its control plane operations, and as many Durable Object instances as we need for the data plane operations, one for each resource instance created in the application.

You can scale to millions of Durable Object instances, one for each of your resources.

The main advantage of this architectural pattern is that our data plane operations, usually with larger volume of requests than control plane operations, are handled directly by the Durable Object instances holding the resource data without going through the control plane Durable Object instance. Therefore, the application's performance and availability is not limited by a single Durable Object instance, but is shared across thousands or millions of Durable Objects.

Consider an example for a generic resource type XYZ, where XYZ could in-practice be a wiki, a collaborative document, a database for each user, or any other resource type in your application.

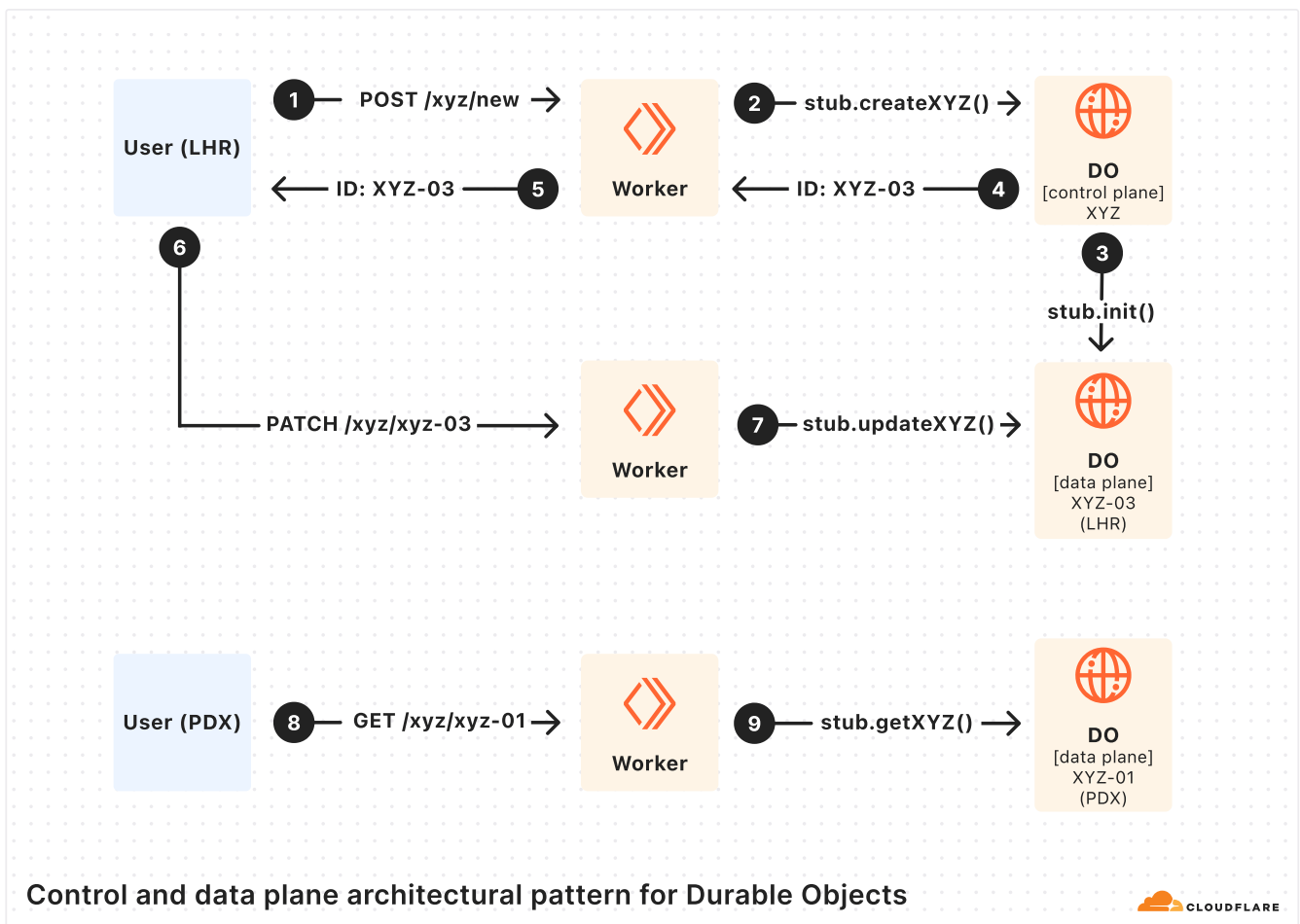


Figure 1: Control and data plane architectural pattern for Durable Objects

1. A user in London (LHR) initiates a resource XYZ creation request. The request is routed to the nearest Cloudflare datacenter and received by the Workers fleet which serves the application API.
2. The Worker code will route the request to the appropriate control plane Durable Object instance managing the resources of type XYZ. We

will use the `idFromName` approach to reference the Durable Object instance by name (`control-plane-xyz`). This allows immediate access to the control plane Durable Object instances without needing to maintain a mapping.

- The location of the control plane Durable Object will be close to the first request accessing it, or to the explicit region we provide using [Location Hints](#).

3. The control plane Durable Object instance (`control-plane-xyz`) receives the request, and immediately creates another Durable Object instance (`data-plane-xyz-03`) near the user request's location (using Location Hints) so that the actual Durable Object instance holding the resource's content is near the user that created it.
 - We call a custom `init(...)` function on the created Durable Object instance (`data-plane-xyz-03`) passing any required metadata info that will be needed to start handling user requests.The Durable Object instance stores this

information in its local storage and performs any necessary initialisation. This step can be skipped if each subsequent request to the created resource contains all the information needed to handle the request. For example, if the request URL contains all the information as path and query parameters. - We use the [idFromName](#) approach to reference the Durable Object (data-plane-xyz-03) which allows the use of name-based resource identifiers. - Alternatively, we can use the [newUniqueId](#) approach to reference the Durable Object which will give us a random resource identifier to use instead of a name-based one. This random identifier will need to be communicated back to the user so that they provide it in their subsequent requests when accessing the resource.

4. The control plane Durable Object instance (control-plane-xyz) stores the generated identifier (data-plane-xyz-03) to its local storage, in order to be able to list/delete all

created resources, and then returns it to the Worker.

5. The user receives a successful response for the creation of the resource and the corresponding identifier, and (optionally) gets redirected to the resource itself.
6. The user sends a write request to the API for the resource identifier returned in the previous step, in order to update the content of the resource.
7. The Worker code uses the resource identifier provided to directly reference the data plane Durable Object instance for that resource (`data-plane-xyz-03`). The Durable Object instance will handle the request appropriately by writing the content to its local durable persistent storage and return a response accordingly.
8. Another user from Portland (PDX) is sending a read request to a previously created resource (`data-plane-xyz-01`).
9. The Worker code directly references the Durable Object instance holding the data for the given

resource identifier (data-plane-xyz-01), and the Durable Object instance will return its content by reading its local storage.

As long as the application data model allows sharding at the resource level, you can scale out as much as you want, while taking advantage of data locality near the user that accesses that resource.

The same pattern can be applied as many times as necessary to achieve the performance required.

For example, depending on our load, we could further shard our control plane Durable Object into several Durable Objects. Instead of having a single Durable Object instance for all resources of type XYZ, we could have one for each region. The name-based approach to reference a Durable Object instance simplifies targeting the appropriate instance accordingly.

In conclusion, as long as you find a way to shard your application's data model in fine-grained resources that are self-contained, you are able to dedicate at least

one Durable Object instance to each resource and scale out.

Related resources

- [Durable Objects Namespace documentation](#)
- [Durable Objects: Easy, Fast, Correct — Choose three ↗](#)
- [Zero-latency SQLite storage in every Durable Object ↗](#)
- [Data, Control, Management: Three Planes, Different Altitudes ↗](#)
- Examples of this architectural pattern in real-world applications:
 - [Durable Objects aren't just durable, they're fast: a 10x speedup for Cloudflare Queues ↗](#)
 - [Building a global TiddlyWiki hosting platform with Cloudflare Durable Objects and Workers — Tiddlyflare ↗](#)

Was this helpful?



Previous

Next

Serverless image
content
management

Egress-free object
storage in multi-cloud
setups



Last updated: Aug 19, 2025

Resources

[API](#)

[New to Cloudflare?](#)

[Directory](#)

[Sponsorships](#)

[Open Source](#)

Support

[Help Center](#)

[System Status](#)

[Compliance](#)

[GDPR](#)

Company

[cloudflare.com](#)

[Our team](#)

Tools

[Cloudflare Radar](#)

[Speed Test](#)

Careers

Is BGP Safe Yet?

RPKI Toolkit

Certificate Transparency

Community

 X

 Discord

 YouTube

 GitHub

© 2025 Cloudflare, Inc. • [Privacy Policy](#) • [Terms of Use](#)

• [Report Security Issues](#) • [Trademark](#) •  [Cookie Preferences](#)